Cracking Query Bottlenecks: Towards Efficiency-Oriented Text-to-SQL Generation

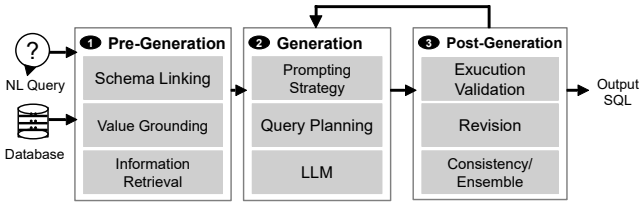

Fig. 1: **Overview of a typical Text-to-SQL pipeline, illustrating the Pre-Generation, Generation, and Post-Generation stages**

***Abstract*—Pending...**

I. Introduction

Text-to-SQL aims to translate natural language (NL) questions into executable SQL queries, enabling non-expert users to retrieve and analyze relational data without manually writing database programs [1], [2], [3]. Its importance has grown with increasingly large schemas, rich database contents, and diverse analytical tasks, motivating benchmarks such as SPIDER [4], BIRD [5]. Recent LLM-based systems have achieved substantial progress through in-context learning, task decomposition and specialized agents, moving the field beyond single-shot generation toward workflow-based SQL construction [6], [7], [8], [9].

From Single-shot Generation to Agentic Workflows. As shown in Figure 1, modern Text-to-SQL systems typically organize SQL construction into three stages: pre-generation, generation, and post-generation. In the pre-generation stage, the system grounds the natural language query in the database context through schema linking, value grounding, and information retrieval, thereby reducing the search space and enriching the input evidence [9], [10], [11], [12]. In the generation stage, the model applies prompting strategies, query planning, and LLM-based reasoning to produce one or more candidate SQL queries [6], [7], [8], [13], [14], [15]. In the post-generation stage, these candidates are further checked and refined through execution validation, repair, and consistency- or ensemble-based selection [16], [7], [17], [8], [9], [13], [14], [15]. Compared with single-pass generation, such workflows are more robust because they decompose SQL construction into modular steps, strengthen database grounding, diversify candidate generation, and provide post-hoc opportunities for validation and refinement. Despite these advances, existing Text-to-SQL systems still face three coupled limitations:

L1: Limited Adaptation in Workflows and Strategy Selection. Many existing Text-to-SQL agents often predefine both the workflow structure and the strategy choices, where modules such as schema linking, few-shot retrieval, question decomposition, candidate generation, validation, and revision are applied in a largely fixed manner [6], [7], [8], [13], [11]. Although such designs improve robustness over single-shot generation, they provide limited support for adapting reasoning strategies to the current database environment, question difficulty, or intermediate feedback. This can misallocate reasoning effort: simple queries over intuitive schemas may incur unnecessary token and latency overhead, while difficult queries may require deeper grounding, candidate comparison, or additional validation and reflection. This mismatch is illustrated by the different effects of domain-specific hints in BIRD [5]. For an intuitive database such as `superhero`, where tables and columns are relatively descriptive, adding domain hints provides little additional benefit in our experiments. In contrast, for a domain-heavy database such as `thrombosis_prediction`, where terms such as `LDH` and gender-dependent uric-acid ranges require medical knowledge, the same hints improve accuracy from 20% to nearly 50%. This contrast suggests that Text-to-SQL agents should adapt their grounding and generation strategies to the database environment, rather than applying the same strategy uniformly.

L2: Post-hoc rather than Process-level Reflection. Feedback in many Text-to-SQL systems is mainly used after a candidate SQL has been generated, such as repairing execution errors, revising invalid queries, or selecting among completed candidates [7], [8], [9], [18]. Such feedback improves final-output validation, but provides limited support for monitoring and regulating the generation process itself. In particular, the agent often lacks a controller that continuously tracks whether schema linking is reliable, whether candidate disagreement is increasing, whether repeated repairs are converging, or whether the current strategy should be changed. As a result, early mistakes in schema linking, value grounding, or query planning may propagate into later steps and become cascading hallucinations. Moreover, without global process supervision, local repair actions may fix surface-level execution errors while leaving the overall reasoning trajectory insufficiently grounded in the user question and database context.

L3: Limited Awareness of Query Efficiency. Most existing Text-to-SQL systems are primarily optimized for correctness: they aim to generate SQL queries whose execution results match the ground-truth answers [19], [20], [21], [22], [23], [24], [6], [7]. However, in practical database applications, a semantically correct SQL query is not necessarily efficient. Equivalent or near-equivalent SQL formulations may lead to

very different execution costs due to redundant projections, unnecessary joins, correlated subqueries, missing filter pushdown, suboptimal join orders, or functions applied to indexed columns [25], [26], [27]. Although database systems have long relied on rule-based, cost-based, and learned rewriting to improve query efficiency [28], [27], [29], [30], current Text-to-SQL agents usually use database feedback mainly to check executability or result consistency. They rarely treat execution plans, estimated costs, or bottleneck patterns as signals for deciding whether and how to rewrite the generated SQL. As a result, the final query may be correct but unnecessarily expensive, especially in enterprise settings with large schemas, large tables, and long multi-step analytical workflows [18].

Key Insights: Meta-cognitive and Environment-aware Text-to-SQL. Inspired by enactive cognition and meta-cognition, we view Text-to-SQL not as a static translation from question and schema to SQL, but as an adaptive interaction process between the agent and the database environment [31], [32]. Enactive cognition suggests that understanding is shaped through interaction with the environment, while meta-cognition emphasizes the ability to monitor and regulate one's own reasoning process. In the context of Text-to-SQL, this means that an agent should perceive database-specific signals, act by generating and rewriting SQL, observe feedback from execution and query plans, and adjust its strategy when uncertainty, failure, or inefficiency is detected. These perspectives lead to three design objectives. First, to address **L1**, the agent should perform *environment-aware strategy selection*: it should select lightweight or deep workflows according to the perceived database environment, question difficulty, and intermediate feedback, rather than applying the same strategy uniformly. Second, to address **L2**, the agent should support *process-level self-reflection*: it should monitor schema-linking reliability, candidate disagreement, repair convergence, and strategy suitability during generation, so that early mistakes can be corrected before they propagate. Third, to address **L3**, the agent should enable *plan-aware SQL optimization*: it should use execution plans, cost signals, semantic validation, and reusable rewrite knowledge to improve query efficiency while preserving correctness.

II. Design Objective

A. Task Formulation

Traditional Text-to-SQL aims to translate a NL question x into an executable SQL query q over a database environment $\mathcal{D}$. Given a ground-truth query q^*, the standard objective is to generate a query whose execution result matches that of q^*:

$$f_\theta(x, \mathcal{D}) \to q, \quad \text{s.t.} \quad \text{Exec}(q, \mathcal{D}) = \text{Exec}(q^*, \mathcal{D}),$$

where f_θ denotes a Text-to-SQL model parameterized by θ, $\mathcal{D}$ denotes the database environment including schema, data, indexes, and statistics, and $\text{Exec}(q, \mathcal{D})$ denotes the result of executing q on $\mathcal{D}$.

However, this formulation focuses mainly on semantic correctness and does not explicitly consider query efficiency. In practice, multiple semantically equivalent SQL queries may produce the same result but incur substantially different execution costs due to join order, predicate placement, aggregation strategy, or index utilization. We therefore formulate **Efficiency-Oriented Text-to-SQL**, where the goal is to generate a query that is both semantically correct and efficient. Given a NL question x and a database environment $\mathcal{D}$, the goal is to generate a SQL query q_{opt} satisfying:

$$q_{\text{opt}} = \arg \min_{q \in \mathcal{Q}_{\text{corr}}(x,\mathcal{D})} \text{Cost}(q, \mathcal{D}),$$

$$\mathcal{Q}_{\text{corr}}(x, \mathcal{D}) = \{q \in \mathcal{Q}(x, \mathcal{D}) \mid \text{Exec}(q, \mathcal{D}) = \text{Exec}(q^*, \mathcal{D})\},$$

where $\mathcal{Q}_{\text{corr}}(x, \mathcal{D})$ denotes the subset of candidates that are semantically correct with respect to q^*. $\text{Cost}(q, \mathcal{S})$ represents the estimated or actual execution cost of query q on $\mathcal{S}$. Intuitively, the model should not only ensure that q_{opt} yields correct results but also that it achieves minimal execution time or resource consumption under the same semantic constraints.

III. OptSQL

IV. Experiment

V. Discussion

VI. Related Work

Fine-tuned Text-to-SQL Models. Early neural Text-to-SQL research mainly formulated the task as supervised semantic parsing over benchmark datasets such as SPIDER [4]. A major line of work improves model architecture and decoding constraints through explicit schema encoding, relation-aware linking, grammar- or parser-guided decoding, and structure-aware pretraining. Representative systems include RAT-SQL [24], which models relations among question tokens and schema elements; PICARD [23], which constrains autoregressive decoding through incremental parsing; and RESDSQL [19] and GRAPHIX-T5 [21], which further improve schema linking, skeleton parsing, and graph-aware representation learning. Reinforcement-learning-based methods such as REASONING-SQL and ARCTIC-TEXT2SQL-R1 further train models with SQL-tailored rewards or execution-based feedback to improve reasoning and executability [33], [34]. These methods substantially advanced cross-domain Text-to-SQL, but their effectiveness often depends on task-specific training data, synthetic data quality, and benchmark distributions.

LLM Prompting and Retrieval-augmented Generation. With the emergence of general-purpose LLMs, Text-to-SQL research has increasingly shifted from model fine-tuning to prompting-based generation. Prompting methods avoid task-specific parameter updates and instead rely on schema serialization, demonstrations, reasoning instructions, and execution feedback. Benchmark studies show that LLMs can serve as strong Text-to-SQL generators when equipped with carefully designed prompts and database context [6], [5]. DIN-SQL [7] decomposes Text-to-SQL into subproblems and uses self-correction to improve generation quality, while SQL-PALM [35] investigates LLM adaptation and prompting strategies for Text-to-SQL. Other approaches improve grounding

through retrieval and context construction, such as retrieval-augmented generation [10] and CHESS [9], which performs contextual harnessing, schema pruning, and candidate selection for efficient SQL synthesis. These methods reduce the need for task-specific fine-tuning, but they still commonly organize generation around predefined stages such as context construction, candidate generation, validation, and selection.

Agentic Text-to-SQL workflows. Recent work further extends LLM prompting into agentic workflows, where LLMs call tools, inspect database context, execute SQL, observe feedback, and revise intermediate outputs. General agent frameworks such as REACT [17], SELF-REFINE [36], and REFLEXION [37] demonstrate the value of combining reasoning, acting, and iterative self-improvement. In Text-to-SQL, systems such as MAC-SQL, CHASE-SQL, ReFoRCE, SQL-of-Thought, AGENTIQL, and MARS-SQL decompose the task into modules for schema grounding, candidate generation, self-refinement, voting or consensus, and execution-based validation [8], [38], [39], [13], [14], [15]. However, most existing agents still instantiate a largely fixed workflow: similar modules are invoked for most questions, and database feedback is primarily used for post-generation repair or selection. In contrast, OPTSQL targets efficiency-oriented Text-to-SQL by introducing a meta-cognitive controller that adapts the workflow to question difficulty, database evidence, execution behavior, and optimization progress.

VII. CONCLUSION

REFERENCES

[1] I. Androutsopoulos, G. D. Ritchie, and P. Thanisch, "Natural language interfaces to databases–an introduction," *Natural language engineering*, vol. 1, no. 1, pp. 29–81, 1995.

[2] G. G. Hendrix, E. D. Sacerdoti, D. Sagalowicz, and J. Slocum, "Developing a natural language interface to complex data," *ACM Transactions on Database Systems (TODS)*, vol. 3, no. 2, pp. 105–147, 1978.

[3] A.-M. Popescu, O. Etzioni, and H. Kautz, "Towards a theory of natural language interfaces to databases," in *Proceedings of the 8th international conference on Intelligent user interfaces*, 2003, pp. 149–157.

[4] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," *arXiv preprint arXiv:1809.08887*, 2018.

[5] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *Advances in Neural Information Processing Systems*, vol. 36, pp. 42 330–42 357, 2023.

[6] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *arXiv preprint arXiv:2308.15363*, 2023.

[7] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," *Advances in Neural Information Processing Systems*, vol. 36, pp. 36 339–36 348, 2023.

[8] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun *et al.*, "Mac-sql: A multi-agent collaborative framework for text-to-sql," *arXiv preprint arXiv:2312.11242*, 2023.

[9] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "Chess: Contextual harnessing for efficient sql synthesis," *arXiv preprint arXiv:2405.16755*, 2024.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[11] M. T. Pham, T. Pham, T. Chen, H. Yin, Q. V. H. Nguyen, and T. T. Nguyen, "Av-sql: Decomposing complex text-to-sql queries with agentic views," *arXiv preprint arXiv:2604.07041*, 2026.

[12] A. Biswal, C. Lei, X. Qin, A. Li, B. Narayanaswamy, and T. Kraska, "Agentsm: Semantic memory for agentic text-to-sql," *arXiv preprint arXiv:2601.15709*, 2026.

[13] S. Chaturvedi, A. Chadha, and L. Bindschaedler, "Sql-of-thought: Multi-agentic text-to-sql with guided error correction," *arXiv preprint arXiv:2509.00581*, 2025.

[14] O. R. Heidari, S. Reid, and Y. Yaakoubi, "Agentiql: An agent-inspired multi-expert framework for text-to-sql generation," *arXiv preprint arXiv:2510.10661*, 2025.

[15] H. Yang, J. Zhang, Z. He, and Y. R. Fung, "Mars-sql: A multi-agent reinforcement learning framework for text-to-sql," *arXiv preprint arXiv:2511.01008*, 2025.

[16] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[18] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows," *arXiv preprint arXiv:2411.07763*, 2024.

[19] H. Li, J. Zhang, C. Li, and H. Chen, "Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 067–13 075.

[20] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "Codes: Towards building open-source language models for text-to-sql," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–28, 2024.

[21] J. Li, B. Hui, R. Cheng, B. Qin, C. Ma, N. Huo, F. Huang, W. Du, L. Si, and Y. Li, "Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 11, 2023, pp. 13 076–13 084.

[22] D. Rai, B. Wang, Y. Zhou, and Z. Yao, "Improving generalization in language model-based text-to-sql semantic parsing: Two simple semantic boundary-based techniques," *arXiv preprint arXiv:2305.17378*, 2023.

[23] T. Scholak, N. Schucher, and D. Bahdanau, "Picard: Parsing incrementally for constrained auto-regressive decoding from language models," *arXiv preprint arXiv:2109.05093*, 2021.

[24] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers," *arXiv preprint arXiv:1911.04942*, 2019.

[25] S. Chaudhuri, "An overview of query optimization in relational systems," in *Proceedings of the seventeenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*, 1998, pp. 34–43.

[26] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[27] Z. Wang, Z. Zhou, Y. Yang, H. Ding, G. Hu, D. Ding, C. Tang, H. Chen, and J. Li, "Wetune: Automatic discovery and verification of query rewrite rules," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 94–107.

[28] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, 1979, pp. 23–34.

[29] X. Zhou, G. Li, J. Wu, J. Liu, Z. Sun, and X. Zhang, "A learned query rewrite system," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 4110–4113, 2023.

[30] Z. Li, H. Yuan, H. Wang, G. Cong, and L. Bing, "Llm-r2: A large language model enhanced rule-based rewrite system for boosting query efficiency," *arXiv preprint arXiv:2404.12872*, 2024.

[31] F. J. Varela, E. Thompson, and E. Rosch, *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press, 1991.

[32] J. H. Flavell, "Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry," *American Psychologist*, vol. 34, no. 10, pp. 906–911, 1979.

[33] M. Pourreza, S. Talaei, R. Sun, X. Wan, H. Li, A. Mirhoseini, A. Saberi, S. O. Arik *et al.*, "Reasoning-sql: Reinforcement learning with sql tailored partial rewards for reasoning-enhanced text-to-sql," *arXiv preprint arXiv:2503.23157*, 2025.

[34] Z. Yao, G. Sun, L. Borchmann, Z. Shen, M. Deng, B. Zhai, H. Zhang, A. Li, and Y. He, "Arctic-text2sql-r1: Simple rewards, strong reasoning in text-to-sql," *arXiv preprint arXiv:2505.20315*, 2025.

[35] R. Sun, S. Ö. Arik, A. Muzio, L. Miculicich, S. Gundabathula, P. Yin, H. Dai, H. Nakhost, R. Sinha, Z. Wang *et al.*, "Sql-palm: Improved large language model adaptation for text-to-sql (extended)," *arXiv preprint arXiv:2306.00739*, 2023.

[36] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 534–46 594, 2023.

[37] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.

[38] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Özcan, and S. O. Arik, "Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql," *arXiv preprint arXiv:2410.01943*, 2024.

[39] M. Deng, A. Ramachandran, C. Xu, L. Hu, Z. Yao, A. Datta, and H. Zhang, "Reforce: A text-to-sql agent with self-refinement, consensus enforcement, and column exploration," *arXiv preprint arXiv:2502.00675*, 2025.